

StreamlitとGoogle Geminiで作る

対話型AIチャットボット

実践的なチャットアプリケーション開発

目次

1. Streamlitとは
2. チャットUIの基本構成要素
3. セッション状態管理 (st.session_state)
4. Google AI Studio (Gemini API) との連携
5. チャットボットの実装
6. チャット履歴の永続化
7. UIの改善テクニック

Streamlitとは

- **Python**だけでWebアプリが作れるフレームワーク
- HTML/CSS/JavaScriptの知識不要
- 2023年にチャットコンポーネントを追加
- データサイエンティストに人気
- LLMとの相性が抜群

```
import streamlit as st
st.title("Hello, Streamlit!")
```

チャットUIの基本構成要素

2つの主要コンポーネント

1. `st.chat_message` - メッセージ表示用コンテナ
2. `st.chat_input` - ユーザー入力ウィジェット

これらを組み合わせることで、対話型のチャットUIを構築できます。

st.chat_message - メッセージ表示

```
import streamlit as st

# ユーザーメッセージ
with st.chat_message("user"):
    st.write("こんにちは!")
    st.write("質問があります")

# アシスタントメッセージ
with st.chat_message("assistant"):
    st.write("どうぞご質問ください")
```

- `with` ブロック内に任意のStreamlit要素を配置可能
- テキスト、チャート、画像なども表示できる

st.chat_message - パラメータ

name (必須)

- `"user"` または `"human"`: ユーザーアイコン
- `"assistant"` または `"ai"`: ボットアイコン

avatar (オプション)

- 絵文字: `"🤖"`, `"👤"`
- Material Icons: `:material/smart_toy:`
- 画像URL: `"https://example.com/avatar.png"`
- ローカルファイル: `"./avatar.png"`

st.chat_input - ユーザー入力

```
# 基本的な使い方
prompt = st.chat_input("メッセージを入力してください")
if prompt:
    st.write(f"あなたの入力: {prompt}")
```

主要パラメータ

- `placeholder`: 入力ヒント
- `max_chars`: 最大文字数制限
- `accept_file`: ファイルアップロード対応
- `disabled`: 入力の無効化

セッション状態管理の重要性

問題：Streamlitの実行モデル

- ユーザーの操作ごとにスクリプト全体が**再実行**される
- 変数は毎回リセットされる
- チャット履歴が消えてしまう！

解決策：st.session_state

- セッション間でデータを保持
- スクリプトの再実行をまたいで永続化

st.session_state - 基本的な使い方

```
import streamlit as st
# 初回のみ初期化（重要！）
if "messages" not in st.session_state:
    st.session_state.messages = []
# メッセージの追加
st.session_state.messages.append({
    "role": "user",
    "content": "こんにちは"
})
# メッセージの読み取り
for msg in st.session_state.messages:
    print(msg["role"], msg["content"])
```

チャット履歴の表示パターン

```
import streamlit as st

# 1. セッション状態の初期化
if "messages" not in st.session_state:
    st.session_state.messages = []

# 2. 過去の全メッセージを履歴から再描画
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# 3. ユーザーの新しい入力を待機
if prompt := st.chat_input("何か入力してください"):
    # 処理...
```

エコボットの実装

```
import streamlit as st

if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

if prompt := st.chat_input("何か入力してください"):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    response = f"エコー: {prompt}"
    st.session_state.messages.append({"role": "assistant", "content": response})
    with st.chat_message("assistant"):
        st.markdown(response)
```

Google AI Studio (Gemini API)

Gemini APIとは

- Googleの次世代大規模言語モデル
- 2025年現在、**google-genai** (新SDK) を推奨

APIキーの取得

ステップ1: Google AI Studioにアクセス

1. <https://aistudio.google.com/apikey> にアクセス
2. Googleアカウントでログイン
3. 「Create API Key」をクリック
4. APIキーをコピーして安全に保管

ステップ2: APIキーの安全な保存

- `.streamlit/secrets.toml` ファイルに保存（推奨）
- 環境変数として設定

APIキーの設定方法

方法1: secrets.toml (推奨)

`.streamlit/secrets.toml` を作成：

```
GEMINI_API_KEY = "your_api_key_here"
```

重要: `.gitignore` に追加すること！

```
import streamlit as st  
api_key = st.secrets["GEMINI_API_KEY"]
```

APIキーの設定方法（続き）

方法2: 環境変数

```
# Linux/Mac
export GEMINI_API_KEY='your_api_key_here'

# Windows (PowerShell)
$env:GEMINI_API_KEY="your_api_key_here"
```

```
import os
api_key = os.getenv('GEMINI_API_KEY')
```


新SDK (google-genai) の使い方

インストール

```
pip install google-genai
```

基本的なセットアップ

```
from google import genai
from google.genai import types

# クライアントの初期化
client = genai.Client(api_key=os.getenv('GEMINI_API_KEY'))

# または環境変数を自動検出
client = genai.Client()
```

シンプルなテキスト生成

```
from google import genai
from google.genai import types
import os

client = genai.Client(api_key=os.getenv('GEMINI_API_KEY'))

# 基本的な生成
response = client.models.generate_content(
    model='gemini-flash-lite-latest',
    contents='AIについて簡単に説明してください'
)
print(response.text)
```

パラメータ付きの生成

```
from google.genai import types

response = client.models.generate_content(
    model='gemini-flash-lite-latest',
    contents='気候変動について教えてください',
    config=types.GenerateContentConfig(
        system_instruction='あなたは親切な科学の専門家です'
    )
)
print(response.text)
```

基本的なGeminiチャットボット

```
import streamlit as st
from google import genai
from google.genai import types

# APIキーの取得
api_key = st.secrets.get("GEMINI_API_KEY")

@st.cache_resource
def init_gemini_client(_api_key):
    return genai.Client(api_key=_api_key)

client = init_gemini_client(api_key)

if "messages" not in st.session_state:
    st.session_state.messages = []

# 続く...
```

基本的なGeminiチャットボット（続き）

```
# メッセージ履歴を表示
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# ユーザー入力
if prompt := st.chat_input("何でも聞いてください..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

# 続く...
```

基本的なGeminiチャットボット（続き2）

```
# アシスタントの応答を生成
with st.chat_message("assistant"):
    # 会話履歴をGemini形式に変換
    contents = []
    for msg in st.session_state.messages:
        role = "user" if msg["role"] == "user" else "model"
        contents.append(types.Content(
            role=role,
            parts=[types.Part(text=msg["content"])]
        ))
    response = client.models.generate_content(
        model='gemini-flash-lite-latest',
        contents=contents,
        config=types.GenerateContentConfig()
    )
    st.markdown(response.text)
    st.session_state.messages.append({
        "role": "assistant", "content": response.text
    })
```

文脈を保持する方法

履歴を手動で管理（今回の実装）

- 毎回全履歴をAPIに送信
- `st.session_state.messages` で履歴を管理
- 会話履歴を `types.Content` のリストに変換
- シンプルで分かりやすい

ポイント

- ユーザーメッセージは `role="user"`
- AIメッセージは `role="model"`

ストリーミング対応チャットボット

ユーザー体験を向上させるストリーミング表示：

```
# プレースホルダーを用意
message_placeholder = st.empty()
full_response = ""

# ストリーミング生成
for chunk in client.models.generate_content_stream(
    model='gemini-flash-lite-latest',
    contents=contents,
    config=types.GenerateContentConfig()
):
    if chunk.text:
        full_response += chunk.text
        # リアルタイムで部分表示
        message_placeholder.markdown(full_response)
```


チャット履歴の保存方法

手法1: セッション内状態 (`st.session_state`)

- 永続性: 揮発性 (ブラウザ更新時に消失)
- 用途: シンプルなチャットボット、プロトタイピング
- 実装コスト: 低

手法2: SQLiteデータベース

- 永続性: 完全永続化 (アプリ再起動後も保持)
- 用途: 本格的なWebアプリ、履歴管理が必要な場合
- 実装コスト: 中

手法1: st.session_state (基本)

```
import streamlit as st

# セッション状態の初期化
if "messages" not in st.session_state:
    st.session_state.messages = []

# メッセージの追加
st.session_state.messages.append({
    "role": "user",
    "content": "こんにちは"
})

# メッセージの表示
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
```

手法2: SQLiteデータベース

```
import sqlite3

DB_FILE = "chat_history.db"

def init_database():
    conn = sqlite3.connect(DB_FILE)
    cur = conn.cursor()

    # テーブル作成
    cur.execute("""
        CREATE TABLE IF NOT EXISTS messages (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            role TEXT NOT NULL,
            content TEXT NOT NULL,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()

def save_message(role, content):
    conn = sqlite3.connect(DB_FILE)
    cur = conn.cursor()
    cur.execute("INSERT INTO messages (role, content) VALUES (?, ?)",
                (role, content))
    conn.commit()
    conn.close()
```

手法2: SQLiteデータベース (続き)

```
def load_messages():  
    """データベースからすべてのメッセージを取得"""  
    conn = sqlite3.connect(DB_FILE)  
    cur = conn.cursor()  
    cur.execute("SELECT role, content FROM messages ORDER BY timestamp ASC")  
  
    messages = []  
    for row in cur.fetchall():  
        messages.append({"role": row[0], "content": row[1]})  
  
    conn.close()  
    return messages  
  
# アプリ起動時にDBから読み込み  
if "messages" not in st.session_state:  
    st.session_state.messages = load_messages()  
  
# メッセージ送信時に保存  
save_message("user", prompt)  
save_message("assistant", response)
```

データベース vs st.session_state

項目	st.session_state	SQLite
永続性	ブラウザ更新で消失	完全永続化
実装難易度	簡単	やや複雑
用途	プロトタイプ	本番環境
履歴管理	なし	タイムスタンプ付き
複数セッション	不可	可能

UIの改善テクニック

1. Markdownによる装飾

```
st.markdown("**ユーザー:** こんにちは")  
st.markdown("**AI:** はい、どうぞ")
```

2. 絵文字の活用

```
st.markdown("お役に立てて嬉しいです😊")
```

3. カスタムアバター

```
user_avatar = "👤"  
bot_avatar = "🤖"
```

UIの改善テクニック（続き）

4. ページ設定

```
st.set_page_config(  
    page_title="AI チャットアシスタント",  
    page_icon="🤖",  
    layout="centered",  
    initial_sidebar_state="expanded"  
)
```

5. サイドバーで設定管理

```
with st.sidebar:  
    st.header("⚙️ 設定")  
    temperature = st.slider("創造性", 0.0, 2.0, 0.7, 0.1)  
    max_tokens = st.number_input("最大トークン数", 256, 4096, 2048)
```


UIの改善テクニック（続き2）

6. ローディング状態の表示

```
# シンプルなスピナー
with st.spinner("考え中..."):
    response = generate_response(prompt)

# 詳細なステータス表示
with st.status("処理中...", expanded=True) as status:
    st.write("🔍 入力进行分析中...")
    st.write("☁️ 応答を生成中...")
    response = generate_response(prompt)
    st.write("✅ 完了！")
    status.update(label="完了！", state="complete")
```

ベストプラクティス

エラーハンドリング

```
from google.genai import errors
try:
    response = client.models.generate_content(
        model='gemini-flash-lite-latest',
        contents=contents,
        config=types.GenerateContentConfig()
    )
    st.markdown(response.text)
except errors.ClientError as e:
    error_message = f"APIエラー: {str(e)}"
    st.error(error_message)
except Exception as e:
    st.error(f"予期しないエラー: {str(e)}")
```

ベストプラクティス

入力バリデーション

```
def validate_input(prompt):  
    if not prompt or not prompt.strip():  
        return False, "メッセージを入力してください"  
    if len(prompt) > 4000:  
        return False, "メッセージが長すぎます（最大4000文字）"  
    return True, None  
if prompt := st.chat_input("質問してください"):  
    is_valid, error_message = validate_input(prompt)  
    if not is_valid:  
        st.warning(error_message)  
    else:  
        # 正常な処理を続行  
        process_message(prompt)
```

ベストプラクティス（続き2）

パフォーマンス最適化

```
# キャッシングの活用
@st.cache_resource
def init_gemini_client(_api_key):
    return genai.Client(api_key=_api_key)

# クライアントを再利用
client = init_gemini_client(api_key)

# メッセージ履歴の制限（オプション）
MAX_MESSAGES = 50

if len(st.session_state.messages) >= MAX_MESSAGES:
    st.session_state.messages = st.session_state.messages[-MAX_MESSAGES:]
```

よくある間違い

✗ 間違い1: セッション状態の初期化を忘れる

```
st.session_state.messages = [] # 毎回リセットされる！
```

✓ 正しい方法

```
if "messages" not in st.session_state:  
    st.session_state.messages = []
```

✗ 間違い2: APIキーをコードに直接書く

```
API_KEY = "AIzaSy..." # 危険！
```

✓ 正しい方法

```
api_key = st.secrets["GEMINI_API_KEY"]
```

よくある間違い（続き）

✗ 間違い3: エラーハンドリングを忘れる

```
response = client.models.generate_content(...) # クラッシュする可能性
```

✓ 正しい方法

```
try:  
    response = client.models.generate_content(...)  
except errors.ClientError as e:  
    st.error(f"エラー: {str(e)}")
```

✗ 間違い4: ページ設定の位置が間違っている

```
st.title("My App")  
st.set_page_config(...) # エラー！
```

✓ 正しい方法

```
st.set_page_config(...) # 最初に実行  
st.title("My App")
```


まとめ

重要なポイント

1. **基本機能:** `st.chat_message` と `st.chat_input` で簡単にチャットUIを構築
2. **状態管理:** `st.session_state` で会話履歴を保持
3. **Gemini API:** 新SDK `google-genai` を使用

- 4. **メッセージフロー**: 初期化→表示→入力→処理の順序を守る
- 5. **履歴管理**: `types.Content` と `types.Part` で会話履歴を構築
- 6. **エラーハンドリング**: try-exceptブロックで堅牢に
- 7. **パフォーマンス**: キャッシング、ストリーミングで最適化